CSE 493
LINUX SYSTEM ADMINISTRATION

Unit I

Accessing the command line : Accessing command line using local console, Accessing command line using desktop, Executing commands using bash shell

Managing Files From the Command Line : The Linux File System Hierarchy, Locating Files by Name, Managing Files Using Command-Line Tools, Matching File Names Using Path Name Expansion, Managing Files with Shell Expansion

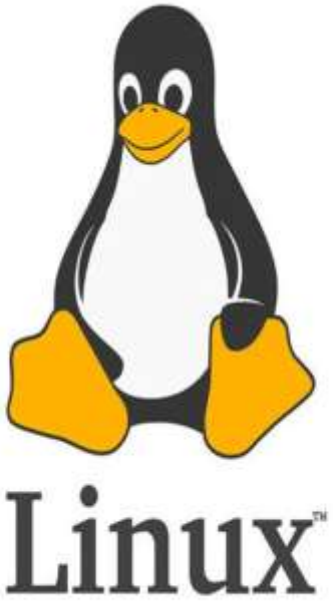
Getting Help in Red Hat Enterprise Linux : Reading Documentation Using man Command,
Reading Documentation Using pinfo Command, Reading Documentation in /usr/share/doc, Getting
Help From Red Hat

Creating, Viewing, and Editing Text Files : Redirecting Output to a File or Program, Editing Text
Files from the Shell Prompt, Editing Text Files with a Graphical Editor

Managing Local Linux Users and Groups : Users and Groups, Gaining Superuser Access,
Managing Local User Accounts, Managing Local Group Accounts, Managing User Passwords

WHAT IS LINUX?

Linux® is an open source operating system (OS) created by Linus Torvalds in 1991. Today, it has a massive user base, and is used in the world's 500 most powerful supercomputers. Users gravitate toward it for its versatility and security capabilities, among other reasons.



Creation

WHY SHOULD YOU LEARN ABOUT LINUX ?

1. You are probably already interacting with Linux systems in your daily life
2. Linux is a critical technology for IT professionals to understand
3. In application development, Linux hosts the application or its runtime.
4. In cloud computing, the cloud instances in the private or public cloud environment use Linux as the operating system.
5. With mobile applications or the Internet of Things (IoT), the chances are high that the operating system of your device uses Linux.
6. If you are looking for new opportunities in IT, Linux skills are in high demand.

WHAT MAKES LINUX GREAT?

Freedom. Power. Flexibility.

1. OPEN SOURCE SOFTWARE

You can see it.
You can change it.
You can share it.

Open source means innovation happens faster.
The community improves it, and everyone benefits.

2. POWERFUL & SCRIPTABLE COMMAND-LINE INTERFACE

Do anything.
Automate everything.
From anywhere.

```
$ ssh user@server
$ ansible-playbook site.yml
$ systemctl enable httpd
$ bash script.sh
```

Built for admins.
Made for automation.
Perfect for scale.

3. MODULAR & FLEXIBLE

Use what you need.
Replace what you want.
Remove what you don't.

Lightweight & minimal

Full-featured & powerful

From a tiny appliance to an enterprise powerhouse—
Linux adapts to you.

LINUX

Built to empower.
Designed to evolve.
Free to innovate.

THAT'S THE POWER OF LINUX. Open. Powerful. Flexible.
It's not just an operating system. It's a movement.

Accessing the command line

OBJECTIVES

After completing this section, you should be able to log in to a Linux system and run simple commands using the shell.

INTRODUCTION TO THE BASH SHELL :

A command line is a text-based interface which can be used to input instructions to a computer system. The Linux command line is provided by a program called the shell

When a shell is used interactively, it displays a string when it is waiting for a command from the user. This is called the shell prompt. When a regular user starts a shell, the default prompt ends with a **$** character, as shown below :

```
[user@host ~]$
```

```
[root@host ~]#
```

Can you Spot the difference over here?

Root User	Regular User
1. Full Access: Can access and modify every file and folder in the system.	**Limited Access:** Can access only their own files and permitted resources.
2. Administrative Tasks: Can install software, create/delete users, and change system settings.	**Cannot perform administrative tasks** unless given permission using sudo.
3. Prompt Symbol: The terminal prompt ends with **#**. Example: root@ubuntu:~#	**Prompt Symbol:** The terminal prompt ends with **$**. Example: User@ubuntu:~$
4. Risk Level: A wrong command can damage the entire operating system.	**Safer:** Mistakes usually affect only the user's own files and account.

SHELL BASICS

Commands entered at the shell prompt have three basic parts:

- Command to run
- Options to adjust the behavior of the command.
- Arguments, which are typically targets of the command

E.g **Usermod -L user01**

LOGGING IN TO A LOCAL COMPUTER

To run the shell, you need to log in to the computer on a terminal. A terminal is a text-based interface used to enter commands into and print output from a computer system. There are several ways to do this.

Physical Console
Directly connected keyboard + monitor plugged into the machine

Virtual Consoles (tty1–tty6)
One physical console → 6 independent login sessions Switch between them: **Ctrl+Alt+F1** to **F6**

Two Types of Logins

Console	Type	What You Get
tty1	Graphical login screen	Desktop environment
tty2–tty6	Text login prompt	Shell prompt directly

LOGGING IN OVER THE NETWORK

Linux users and administrators often need to get shell access to a remote system by connecting to it over the network. In a modern computing environment, many headless servers are actually virtual machines or are running as public or private cloud instances

The most common way to get a shell prompt on a remote system is to use Secure Shell (SSH).

```
[user@host ~]$ ssh remoteuser@remotehost
remoteuser@remotehost's password: password
[remoteuser@remotehost ~]$
```

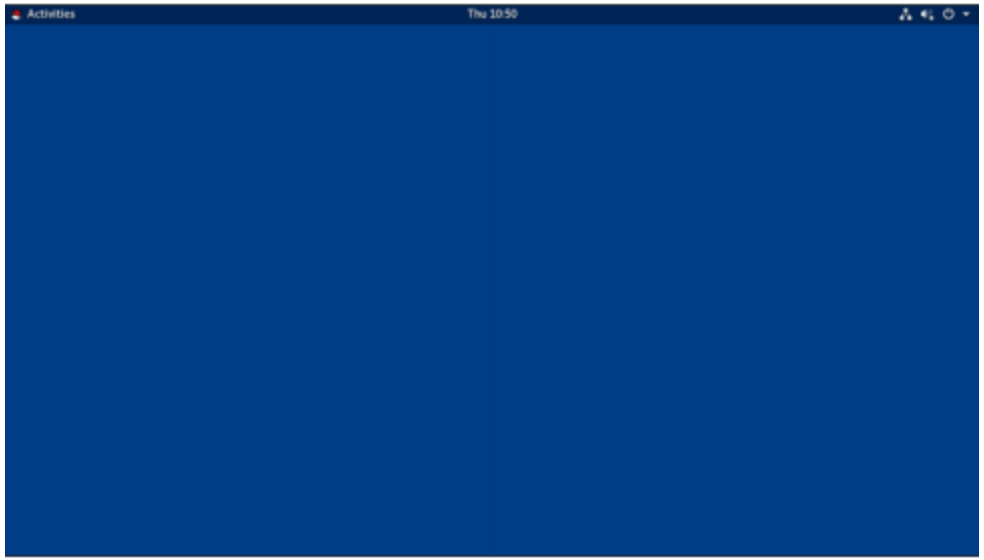
The **ssh** command encrypts the connection to secure the communication against eavesdropping or hijacking of the passwords and content.

LOGGING OUT : When you are finished using the shell and want to quit **Ctrl +D**

ACCESSING THE COMMAND LINE USING THE DESKTOP

The desktop environment is the graphical user interface on a Linux system. The default desktop environment in Red Hat Enterprise Linux 8 is provided by GNOME 3. It provides an integrated desktop for users and a unified development platform on top of a graphical framework provided by either Wayland (by default) or the legacy Window System.

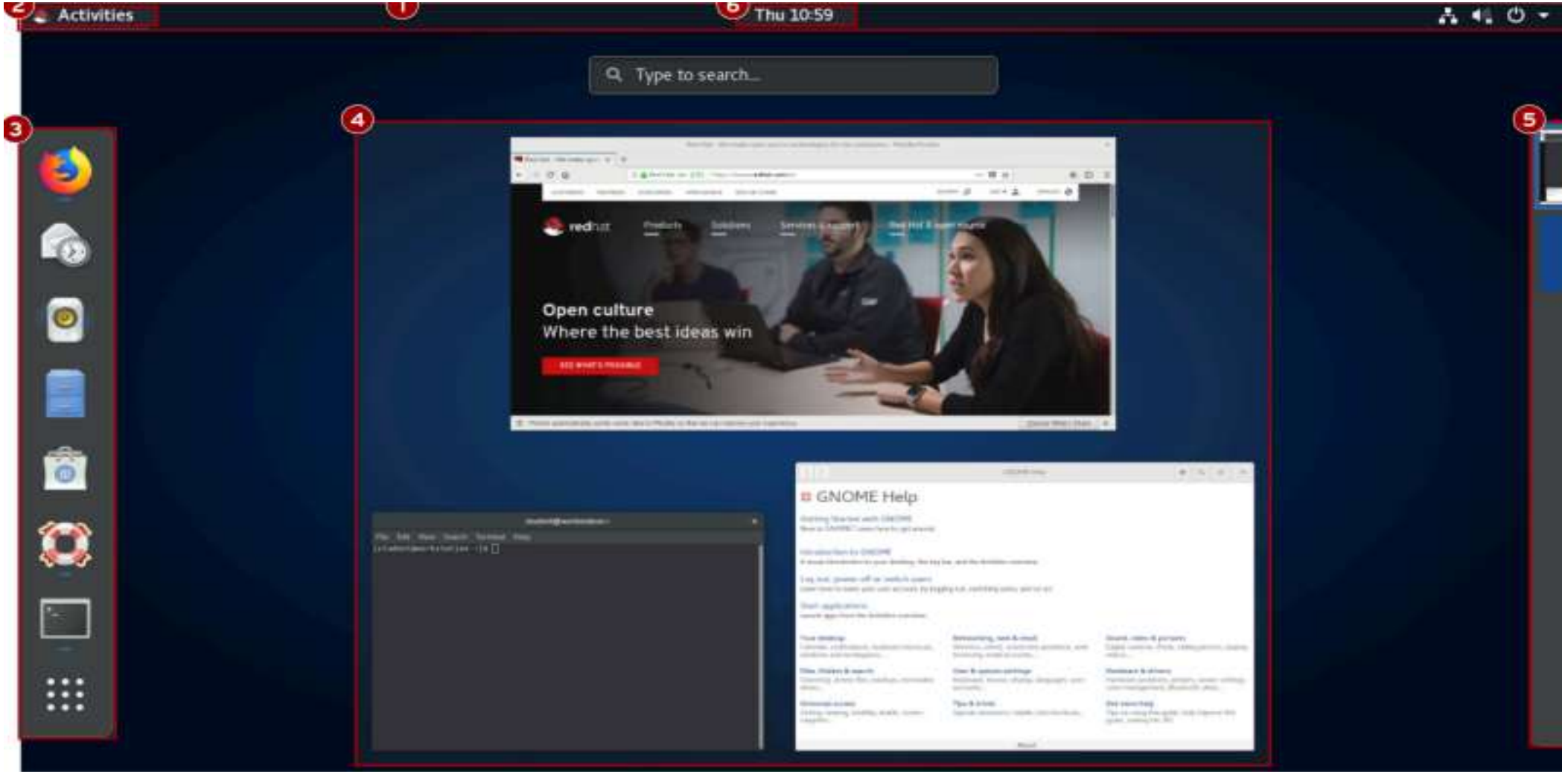
1. **GNOME Shell** is the main interface of the GNOME desktop — and it's customizable.
2. **RHEL 8** uses the **"Standard"** theme by default; **RHEL 7** used **"Classic"** (older GNOME look).
3. To switch themes: click the **gear icon** ⚙ next to "Sign In" — **before** entering your password.
4. Your chosen theme is **saved** and used every time you log in.



Empty GENOME Desktop

You can quickly start GNOME Help by clicking the Activities button on the left side of the top bar, and in the dash that appears on the left side of the screen, clicking the life ring buoy icon to launch it.

ACCESSING THE COMMAND LINE USING THE DESKTOP



ACCESSING THE COMMAND LINE USING THE DESKTOP

1. **Top Bar:** The bar at the top of the screen that provides access to Activities, system status, settings, notifications, and user options.
2. **Activities Overview:** A mode for launching applications, viewing open windows, and managing workspaces.
3. **Dash (Dock):** A sidebar containing favorite apps, running applications, and the Applications Grid button. **Windows Overview:** Displays thumbnails of all open windows in the current workspace for easy switching and organization.
4. **Workspace Selector:** Shows available workspaces and allows you to switch between them or move windows across workspaces.

WORKSPACES

Workspaces are separate desktop screens that have different application windows. These can be used to organize the working environment by grouping open application windows by task. For example, windows being used to perform a particular system maintenance activity (such as setting up a new remote server) can be grouped in one workspace, while email and other communication applications can be grouped in another workspace.

There are two simple methods for switching between workspaces. One method, perhaps the fastest, is to press **Ctrl**+**Alt**+**UpArrow** or **Ctrl**+**Alt**+**DownArrow** to switch between workspaces sequentially. The second is to switch to the Activities overview and click the desired
workspace.

STARTING A TERMINAL

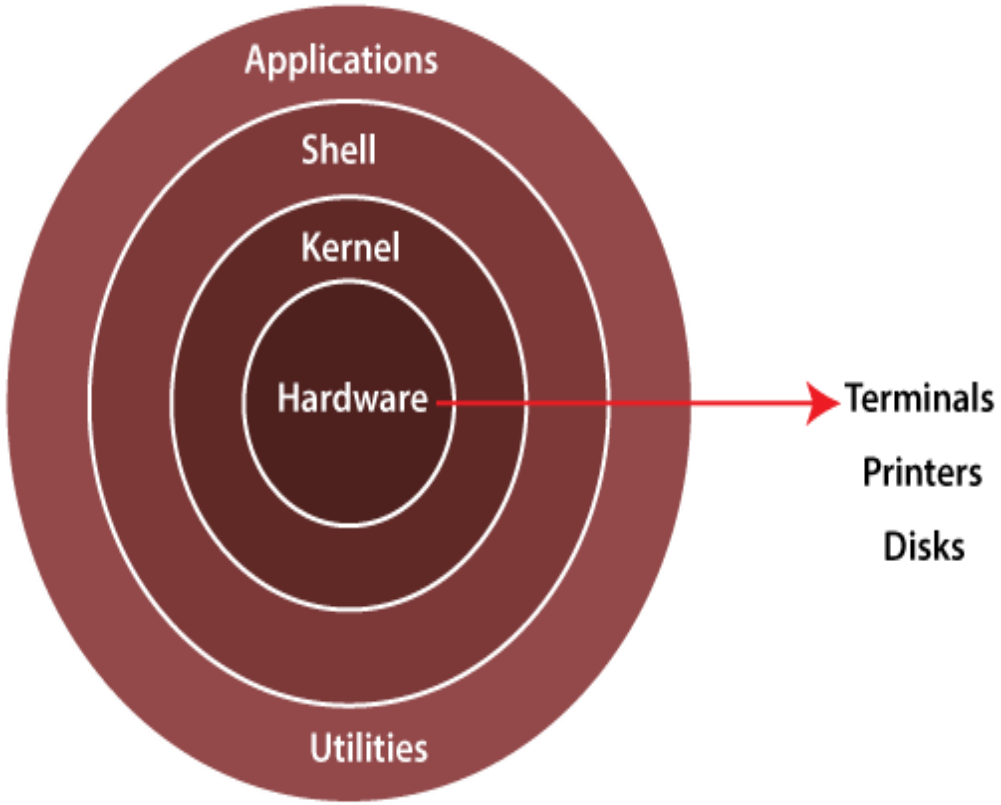
To get a shell prompt in GNOME, start a graphical terminal application such as GNOME Terminal. There are several ways to do this. The two most commonly used methods are listed below:

- From the Activities overview, select Terminal from the dash (either from the favorites area or by finding it with either the grid button (inside Utilities grouping) or the search field at the top of the windows overview).
- Press the **Alt**+**F2** key combination to open the Enter a Command and enter **gnome-terminal**.

LOCKING THE SCREEN OR LOGGING OUT

To lock the screen, from the system menu in the upper-right corner, click the lock button at the bottom of the menu or press **Super**+**L** (which might be easier to remember as **Windows**+**L**).

EXECUTING COMMANDS USING THE BASH SHELL



Linux Architecture

Kernel is the core component of the Linux operating system that sits between the hardware and user space, managing system resources and ensuring smooth communication between software and hardware.

Shell and Utilities

Command interpreter: A user interface (such as Bash) that accepts text commands and passes them along to be executed.

System tools: Small programs used by administrators to configure, monitor, and maintain the system

Programs: The topmost layer where user-facing software, office tools, and graphical or command-line programs run isolated from direct hardware access for enhanced stability

EXECUTING COMMANDS USING THE BASH SHELL

```
[user@host]$ whoami
user
[user@host]$
```

whoami: Displays the username of the currently logged-in user.
Output: user → The current logged-in user's username is **user**.

```
[user@host]$ command1;command2
```

If you want to type more than one command on a single line, use the semicolon (**;**) as a command separator

Simple Example

```
[user@host]$ pwd; date
/home/user
Mon Jul 27 10:45:30 IST 2026
[user@host]$
```

EXECUTING COMMANDS USING THE BASH SHELL

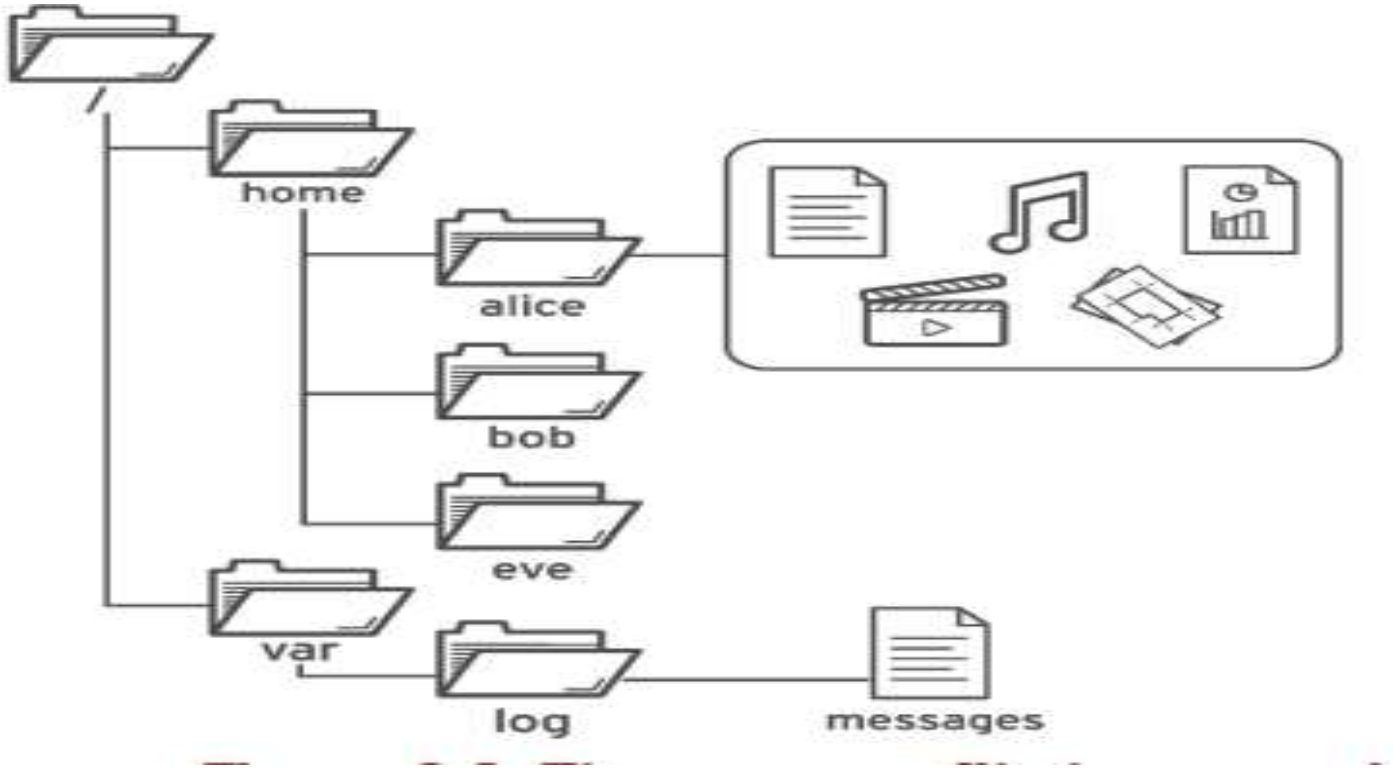
```
[user@host ~]$ date
Sat Jan 26 08:13:50 IST 2019
[user@host ~]$ date +%R
08:13
[user@host ~]$ date +%x
01/26/2019
```

The **date** command displays the current date and time. It can also be used by the superuser to set the system clock. An argument that begins with a plus sign (+) specifies a format string for the date command.

```
[user@host ~]$ passwd
Changing password for user user.
Current password: old_password
New password: new_password
Retype new password: new_password
passwd: all authentication tokens updated successfully.
```

The **passwd** command changes a user's own password. The original password for the account must be specified before a change is allowed. The superuser can use the **passwd** command to change other users' passwords

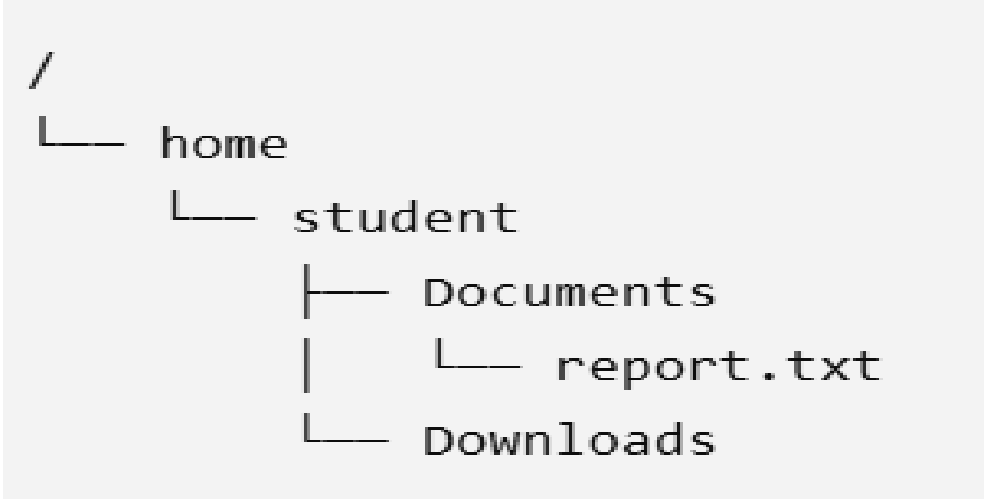
Absolute Path vs Relative Path



Absolute Path vs Relative Path

Feature	Absolute Path	Relative Path
1. Starting Point	Starts from the **root directory (/)**	Starts from your **current working directory**
2. Begins With	Always begins with **/**	Never starts with /. It may start with ./, ../, or a folder name.
3. Depends on Current Location?	No. It works from anywhere in the filesystem.	Yes. It only works correctly if you are in the expected current directory.
4. Example	/home/student/Documents/report.txt	Documents/report.txt, ../Documents/report.txt, ./report.txt

Absolute Path vs Relative Path



Suppose your current directory is Downloads
-> To open report.txt

Absolute Path will be -> cat /home/student/Documents/report.txt

Relative Path will be -> cat ../Documents/report.txt (..) Why?? Because you are currently in Downloads

Frequently used Commands

ls	Lis the contents of the fie	ls ,ls –l , ls -a
cd	Changes the current working directory.	cd /home/user/projects
pwd	Displays the full path of the current working directory.	pwd
mkdir	Creates a new directory.	mkdir my_project
rm	Removes files or directories. Use -r for directories.	rm file.txt or rm -r old_folder
cp	Copies files or directories from one location to another.	cp file.txt backup.txt
mv	Moves or renames files and directories.	mv old.txt new.txt
cat	Displays the contents of a file or combines multiple files.	cat README.md

Command Line

Description

aksh_maverick → Current logged-in user.
AKSHANSH-PC → Computer (host) name.
~ → Current directory is the user's home directory.
$ → Indicates a normal user shell is ready to accept commands.

```
aksh_maverick@AKSHANSH-PC:~$ mkdir schoolai schoolcse schoolece
aksh_maverick@AKSHANSH-PC:~$ ls
3                                  course1    Downloads_backup  myorg-validator.pem  Project1   schoolece  user1
chef-server-core_15.1.7-1_amd64.deb.1  course2    linux_for_devops  newfile.txt          Project2   snap       user2
chef-workstation.deb               Developer  LOPEZ             newuser              Projects   Technova   user3
Classroom1                         devops     love              Practical1           schoolai   test.sh
course                             Downloads  MynewProject      project              schoolcse  Thesis
aksh_maverick@AKSHANSH-PC:~$
```

Command Line

Description

Command/Action	Description
mkdir schoolai schoolcse schoolece	Creates three directories (schoolai, schoolcse, and schoolece) in a single command.
ls	Lists all files and directories in the current working directory (~).

```
aksh_maverick@AKSHANSH-PC:~$ mkdir schoolai schoolcse schoolece
aksh_maverick@AKSHANSH-PC:~$ ls
3                                  course1    Downloads_backup  myorg-validator.pem  Project1   schoolece  user1
chef-server-core_15.1.7-1_amd64.deb.1  course2    linux_for_devops  newfile.txt          Project2   snap       user2
chef-workstation.deb               Developer  LOPEZ             newuser              Projects   Technova   user3
Classroom1                         devops     love              Practical1           schoolai   test.sh
course                             Downloads  MynewProject      project              schoolcse  Thesis
aksh_maverick@AKSHANSH-PC:~$
```

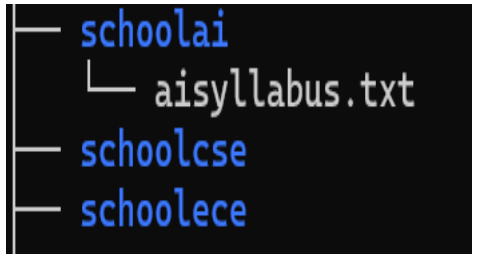
Command Line

Description

Command/Action	Description
mkdir schoolai schoolcse schoolece	Creates three directories (schoolai, schoolcse, and schoolece) in a single command.
ls	Lists all files and directories in the current working directory (~).

```
aksh_maverick@AKSHANSH-PC:~/schoolai$ echo "Below is the attached syllabus " >aisyllabus.txt
aksh_maverick@AKSHANSH-PC:~/schoolai$ cat aisyllabus.txt
Below is the attached syllabus
aksh_maverick@AKSHANSH-PC:~/schoolai$ ls
aisyllabus.txt
aksh_maverick@AKSHANSH-PC:~/schoolai$ cd ..
aksh_maverick@AKSHANSH-PC:~$ tree
```

Tree Structure like display



Command/Action	Description
echo "Below is the attached syllabus" > aisyllabus.txt	Creates a new file named aisyllabus.txt and writes the given text into it, replacing any existing content.
cat aisyllabus.txt	Displays the contents of the aisyllabus.txt file on the terminal, confirming the text was written successfully.
ls and cd ..	ls lists the file in the current directory, while cd .. moves one level up to the parent directory before running tree to view the directory structure.

```
aksh_maverick@AKSHANSH-PC:~$ pwd
/home/aksh_maverick
aksh_maverick@AKSHANSH-PC:~$ cd schoolcse
aksh_maverick@AKSHANSH-PC:~/schoolcse$ touch syllabuscse.txt
aksh_maverick@AKSHANSH-PC:~/schoolcse$ ls
syllabuscse.txt
aksh_maverick@AKSHANSH-PC:~/schoolcse$ touch listofstudents.txt
aksh_maverick@AKSHANSH-PC:~/schoolcse$ ls
listofstudents.txt  syllabuscse.txt
aksh_maverick@AKSHANSH-PC:~/schoolcse$ pwd
/home/aksh_maverick/schoolcse
aksh_maverick@AKSHANSH-PC:~/schoolcse$ cp listofstudents.txt ../schoolai/
aksh_maverick@AKSHANSH-PC:~/schoolcse$ ls
listofstudents.txt  syllabuscse.txt
aksh_maverick@AKSHANSH-PC:~/schoolcse$ cd ..
aksh_maverick@AKSHANSH-PC:~$ cd schoolai
aksh_maverick@AKSHANSH-PC:~/schoolai$ ls
aisyllabus.txt  listofstudents.txt
aksh_maverick@AKSHANSH-PC:~/schoolai$ pwd
/home/aksh_maverick/schoolai
aksh_maverick@AKSHANSH-PC:~/schoolai$
```

Step	Description
1. Navigate and create files	The user checks the current directory using pwd, enters the schoolcse directory, and creates two files: syllabuscse.txt and listofstudents.txt using the touch command.
2. Verify the files	The ls command is used to confirm that both files have been successfully created in the schoolcse directory.
3. Copy file using a relative path	The command cp listofstudents.txt ../schoolai/ copies the listofstudents.txt file from schoolcse to the sibling directory schoolai using the relative path ../schoolai/.
4. Confirm the copy	The user navigates to the schoolai directory and uses ls and pwd to verify that listofstudents.txt has been copied successfully and to display the current directory path.

```
aksh_maverick@AKSHANSH-PC:~$ mkdir FindDemo
cd FindDemo

mkdir Documents Pictures Music

touch Documents/report.txt
touch Documents/notes.txt
touch Pictures/photo.jpg
touch Pictures/logo.png
touch Music/song.mp3
aksh_maverick@AKSHANSH-PC:~/FindDemo$ find . -name "report.txt"
./Documents/report.txt
aksh_maverick@AKSHANSH-PC:~/FindDemo$ find . -name "*.txt"
./Documents/notes.txt
./Documents/report.txt
aksh_maverick@AKSHANSH-PC:~/FindDemo$ find . -type d
.
./Pictures
./Documents
./Music
aksh_maverick@AKSHANSH-PC:~/FindDemo$ find Documents -name "*.txt"
Documents/notes.txt
Documents/report.txt
aksh_maverick@AKSHANSH-PC:~/FindDemo$ ls
Documents  Music  Pictures
aksh_maverick@AKSHANSH-PC:~/FindDemo$ pwd
/home/aksh_maverick/FindDemo
aksh_maverick@AKSHANSH-PC:~/FindDemo$
```

Option	Meaning	Example
.	Search from the current directory.	find . -name "*.txt"
-name	Search by file or directory name.	find . -name "report.txt"
"*.txt"	* matches any number of characters before .txt.	notes.txt, report.txt
-type f	Search only for regular files.	find . -type f
-type d	Search only for directories.	find . -type d

Scenario

You have recently joined **LPU Solutions** as a **Junior Linux System Administrator**.

The company has started a new software development project, but the project workspace is not yet organized. Your team lead has assigned you the responsibility of setting up the directory structure, managing files, organizing departmental data, locating important documents, and cleaning up unnecessary files using only Linux command-line tools.

Task 1

Verify your current working directory and examine the existing contents.

Task 2

Create a directory structure for the following departments:

HR Sales IT Finance Projects

Task 3

Create the required files for each department according to their work responsibilities.

Task 4

Insert appropriate sample data into all text files and verify the contents.

Task 6

Copy selected files from one department to another while keeping the original files unchanged.

Task 7

Locate specific files whose locations are unknown without manually browsing the directory structure.

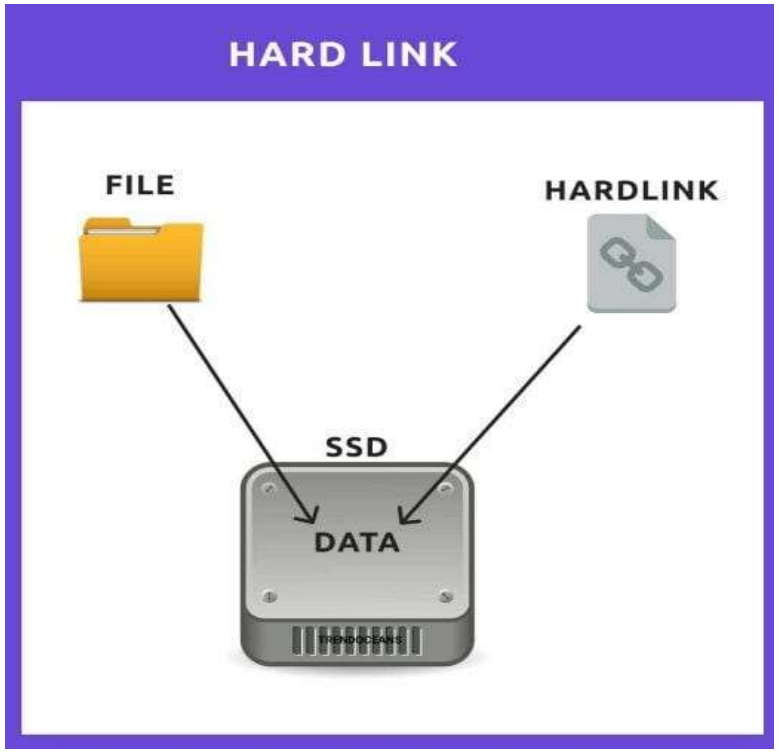
Task 8

Delete unnecessary files, remove empty directories, and clean up obsolete project folders.

What are Hard Links and Soft Links in Linux

Hard Link

The concept of a hard link is the most basic we will discuss today. Every file on the Linux filesystem starts with a single hard link. The *link* is between the filename and the actual data stored on the filesystem



How to Create a Hard Link

aksh_maverick@AKSHANSH-PC: echo “text” >file_name.txt
aksh_maverick@AKSHANSH-PC: ln file_name.txt file_name2.txt

If you edit one file ?

The changes will be reflected in both files

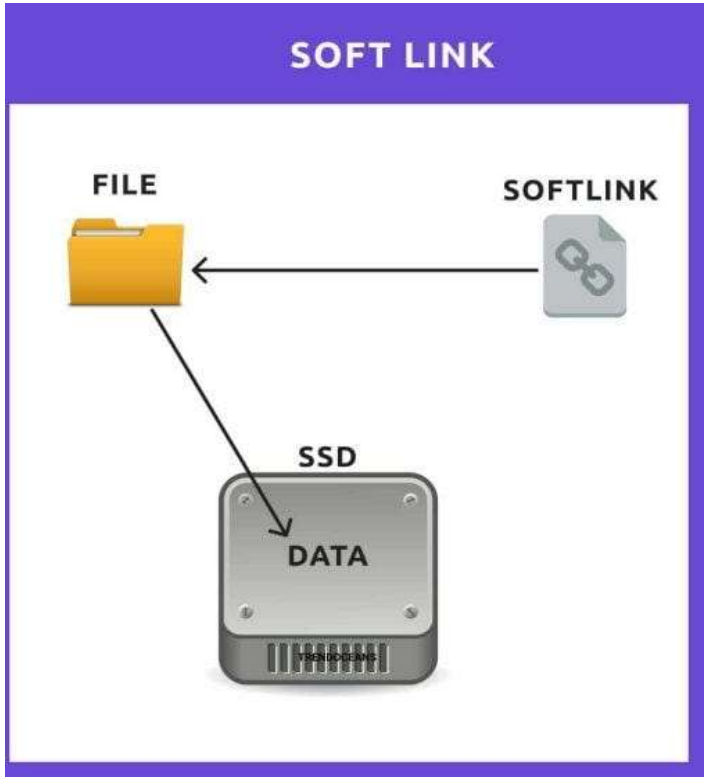
Now comes the concept of Inode

Since both are pointing to the same file they will have same inode number

What are Hard Links and Soft Links in Linux

Soft Link

A **soft link** in Linux is a special file that acts as a shortcut by **pointing to the text path of another file or directory**. Unlike a hard link, it has a unique inode number and can cross different file systems, but it breaks if the original file is deleted (ln -s file_name link_name)



How to Create a soft Link?

```
aksh_maverick@AKSHANSH-PC:~$ echo "This is myfile " > d1.txt
aksh_maverick@AKSHANSH-PC:~$ echo "this is updated file" > d2.txt
aksh_maverick@AKSHANSH-PC:~$ cat d2.txt
this is updated file
aksh_maverick@AKSHANSH-PC:~$ cat d1.txt
this is updated file
aksh_maverick@AKSHANSH-PC:~$ echo "This is my notes1.txt" > notes1.txt
aksh_maverick@AKSHANSH-PC:~$ ln -s notes1.txt shortcut.txt
aksh_maverick@AKSHANSH-PC:~$ cat shortcut.txt
This is my notes1.txt
aksh_maverick@AKSHANSH-PC:~$ rm notes1.txt
aksh_maverick@AKSHANSH-PC:~$ cat shortcut.txt
cat: shortcut.txt: No such file or directory
aksh_maverick@AKSHANSH-PC:~$
```

What is the difference betweem Hard Links and Soft Links in Linux

Feature	Hard Link	Soft Link (Symbolic Link)
Concept	Another **name** for the same file.	A **shortcut** that stores the path to the original file.
If Original File is Deleted	Still works because the data is still accessible through the hard link.	Becomes a **broken (dangling) link** because the target file no longer exists.
Directories	Cannot normally be created for directories.	Can be created for both **files and directories**.
Different File Systems	Cannot be created across different file systems or partitions.	Can be created across different file systems or partitions.

What is Output Redirection?

Normally, every command displays its output on the **terminal**.

What if I want to save its output?

```
aksh_maverick@AKSHANSH-PC:~$ date > date.txt
aksh_maverick@AKSHANSH-PC:~$ cat date.txt
Thu Jul 30 11:13:24 AM IST 2026
aksh_maverick@AKSHANSH-PC:~$
```

Rule 1: > Overwrite

```
aksh_maverick@AKSHANSH-PC:~$ echo "Linux Commands" > a1.txt
aksh_maverick@AKSHANSH-PC:~$ cat a1.txt
Linux Commands
aksh_maverick@AKSHANSH-PC:~$ echo "Linux Commands Updated" > a1.txt
aksh_maverick@AKSHANSH-PC:~$ cat a1.txt
Linux Commands Updated
aksh_maverick@AKSHANSH-PC:~$
```

If the file already exists, its old content is **deleted** and replaced.

What is Output Redirection?

Normally, every command displays its output on the **terminal**.

Rule 2: >> Append

```
aksh_maverick@AKSHANSH-PC:~$ echo "Linux Commands"> a1.txt
aksh_maverick@AKSHANSH-PC:~$ cat a1.txt
Linux Commands
aksh_maverick@AKSHANSH-PC:~$ echo "Linux Commands Updated">>a1.txt
aksh_maverick@AKSHANSH-PC:~$ cat a1.txt
Linux Commands
Linux Commands Updated
aksh_maverick@AKSHANSH-PC:~$
```

Adds new data to the end.

What is Output Redirection?

Normally, every command displays its output on the **terminal**.

Rule 3: 2> Store Errors

```
aksh_maverick@AKSHANSH-PC:~$ ls abc.txt
ls: cannot access 'abc.txt': No such file or directory
aksh_maverick@AKSHANSH-PC:~$ ls abc.txt 2> errors.txt
aksh_maverick@AKSHANSH-PC:~$ cat erros.txt
cat: erros.txt: No such file or directory
aksh_maverick@AKSHANSH-PC:~$
```

This stores errros in a file.

What is Pipeline?

A **pipeline** sends the **output of one command** directly as the **input of another command**. Denoted by symbol **(|)**

Why Do We Need Pipelines?

Scenario : Suppose you have 500 files. Someone asks you Just tell me how many files are there

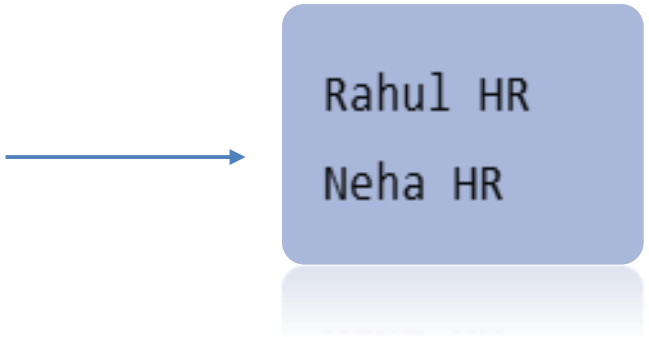
Your Approach
Run Command ls and count manually

Using Pipeline :
ls | wc -l

What is Pipeline?

Find Only HR Employees
Suppose employees.txt contains:
Rahul HR
Priya Sales
Aman IT
Neha HR

Some one Asks?
"Show me only HR employees.“
cat employees.txt | grep HR



Scenario

You have joined **LPU solutions.** as a **Junior Linux System Administrator**. Your Team Lead has assigned you a practical test to evaluate your Linux command-line skills

Part 1 – Hard Link
Create a hard link of employees.txt inside the backup directory.
Verify that both files point to the same inode.

Part 2 - Soft Link
Create a symbolic (soft) link of budget.txt inside the reports directory.
Verify that the symbolic link has been created successfully.

Part 3 – Input and Output Redirection
Save the current date and time into a file named daily_report.txt.
Append the current logged-in user's name to the same file.

Part 4 – Pipelines
Display only the lines containing the word ERROR from server.log.

EDITING TEXT FILES FROM THE SHELL PROMPT

A key design principle of Linux is that information and configuration settings are commonly stored in text-based files. These files can be structured in various ways, as lists of settings, as structured XML or YAML, and so on. However, the advantage of text files is that they can be viewed and edited using any simple text editor

EDITING FILES WITH VIM

1. **Edit configuration files:** Use a text editor directly from a text-only shell to create, modify, and save text-based configuration files.
2. **Work on local or remote systems:** Edit files efficiently from a terminal window or during remote administration using **SSH** or the **Web Console**, even without a graphical interface.

1. Command Mode (Default Mode)

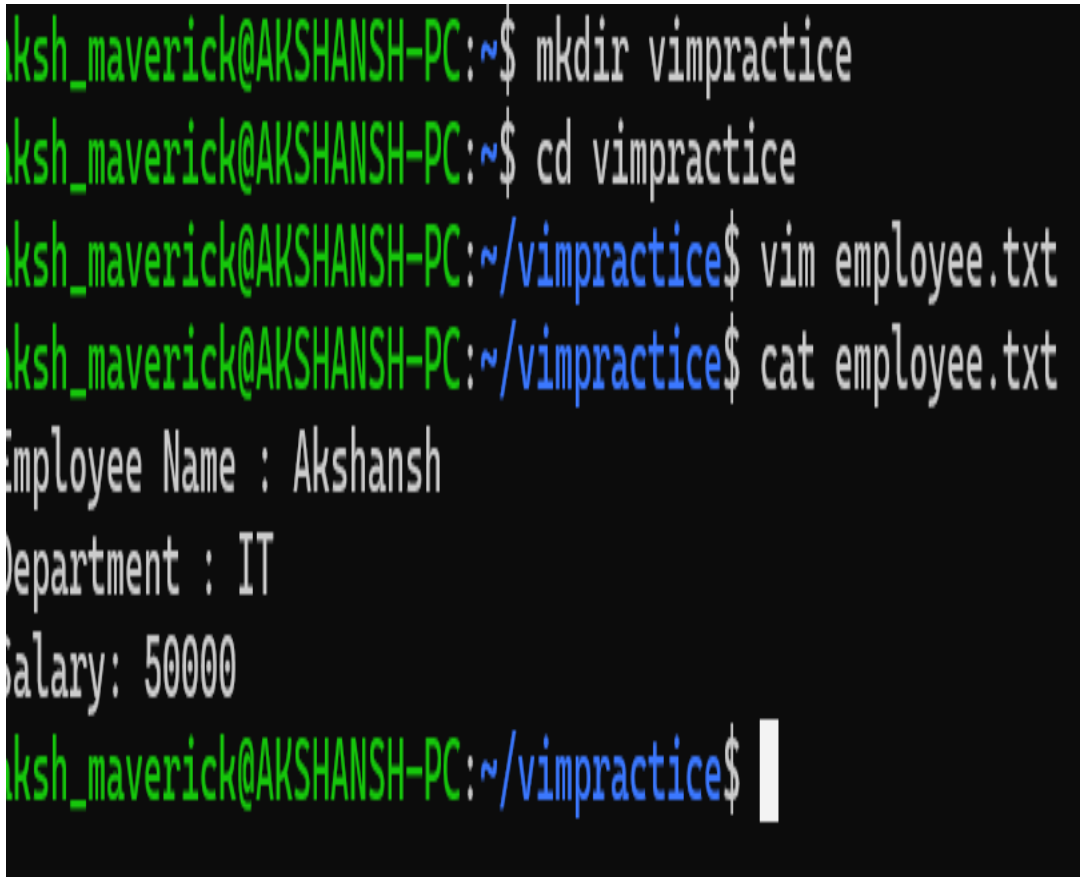
```
aksh_maverick@AKSHANSH-PC:~$ vim notes.txt
```

2. Insert Mode (I)

When you Press I you get to type becomes text**Note – When you finish Press ESC to get back to command mode**

.

3. **Visual Mode (V)**
Then you can
->Copy
->Delete
->Move
Character selection : **Shift +V**
Whole line selection : **Ctrl+V**
4. Extended Command Mode
Save - :w
Quit - :q
Save and quit - :wq



CHANGING THE SHELL ENVIRONMENT

USING SHELL VARIABLES

1. **Shell variables** are used to store values (such as text, paths, or numbers) that can be reused in commands or to change the shell's behavior.
2. They help reduce typing by storing long commands or frequently used values.
3. A **shell variable** exists **only in the current shell session**.
4. If you open another terminal or log in again, it gets a **separate shell** with its **own set of shell variables**.
5. You can **export** a shell variable to make it an **environment variable**, which is automatically passed to programs started from that shell.

Assigning Values to Variables

You can **create a shell variable** by using the syntax:
VARIABLE_NAME=value

Viewing All Shell Variables
set
Since the output is very long, it is commonly used with less
set | less

Shell Variables → Used only by **Bash (the shell)**.
Environment Variables → Used by **Bash and all programs launched from Bash**.

What is a Shell Variable ?
NAME=VARIABLE_NAME
echo $NAME
Output =NAME
Only **Bash** knows this variable

What is an Environment Variable?

Suppose you want **other programs** (like Python, Vim, Git, etc.) to also know this variable.
Then export it.

export NAME ->Syntax

Now every program started from this terminal can also access NAME

Why do we need Environment Variables?

Many programs look for certain variables to decide how they should work.

1. HOME

Run : echo $HOME

/home/name ---- Output

When a program wants to save your settings, it asks:

"Where is the user's home folder?"

It checks the **HOME** variable.

What is an Environment Variable?

2. EDITOR

Suppose a program wants to open a text editor.
How does it know which editor you prefer?
RUN : export EDITOR=vim
Now programs that use the EDITOR variable will open **Vim**

3. PATH

Suppose you type
Run : ls
How does Linux know where ls is
It checks the **PATH** variable.
/usr/local/bin Meaning -> This means:Search for commands in these folders.
/usr/local/bin . Where ls finds it runs it

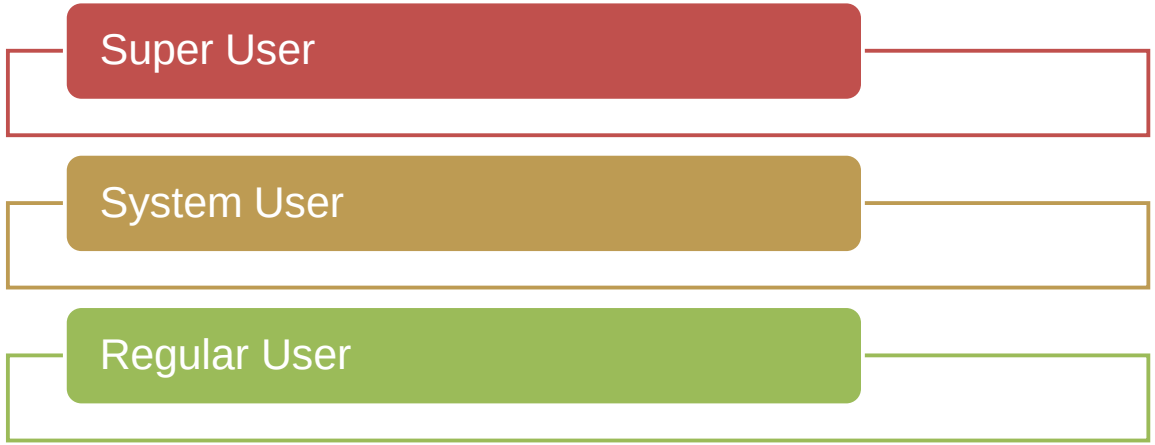
DESCRIBING USER AND GROUP CONCEPTS

WHAT IS A USER?

A user account is used to provide security boundaries between different people and programs that can run commands.

The system distinguishes user accounts by the unique identification number assigned to them, the user ID or UID

Every process (running program) on the system runs as a particular user.



DESCRIBING USER AND GROUP CONCEPTS

```
[user01@host ~]$ id
uid=1000(user01) gid=1000(user01) groups=1000(user01)
```

To view information about the current user that is logged in

```
[user01@host]$ id user02
uid=1002(user02) gid=1001(user02) groups=1001(user02)
```

DESCRIBING USER AND GROUP CONCEPTS

```
[user01@host ~]$ ls -l file1
-rw-rw-r--. 1 user01 user01 0 Feb  5 11:10 file1
[user01@host]$ ls -ld dir1
drwxrwxr-x. 2 user01 user01 6 Feb  5 11:10 dir1
```

To view the owner of a file use the **ls -l** command. To view the owner of a directory use the **ls -** l**d** command

```
[user01@host]$ ps -au
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         777  0.0  0.0 225752  1496 tty1     Ss+  11:03   0:00 /sbin/agetty -o -
p -- \u --noclear tty1 linux
root         780  0.0  0.1 225392  2064 ttyS0    Ss+  11:03   0:00 /sbin/agetty -o -
p -- \u --keep-baud 115200,38400,9600
user01      1207  0.0  0.2 234044  5104 pts/0    Ss   11:09   0:00 -bash
user01      1319  0.0  0.2 266904  3876 pts/0    R+   11:33   0:00 ps au
```

To view process information, use the **ps** command. The default is to show only processes in the current shell. Add the **a** option to view all processes with a terminal. To view the user associated with a process, include the **u** option

DESCRIBING USER AND GROUP CONCEPTS

```
[user01@host ~]$ ls -l file1
-rw-rw-r--. 1 user01 user01 0 Feb  5 11:10 file1
[user01@host]$ ls -ld dir1
drwxrwxr-x. 2 user01 user01 6 Feb  5 11:10 dir1
```

To view the owner of a file use the **ls -l** command. To view the owner of a directory use the **ls -** l**d** command

```
[user01@host]$ ps -au
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         777  0.0  0.0 225752  1496 tty1     Ss+  11:03   0:00 /sbin/agetty -o -
p -- \u --noclear tty1 linux
root         780  0.0  0.1 225392  2064 ttyS0    Ss+  11:03   0:00 /sbin/agetty -o -
p -- \u --keep-baud 115200,38400,9600
user01      1207  0.0  0.2 234044  5104 pts/0    Ss   11:09   0:00 -bash
user01      1319  0.0  0.2 266904  3876 pts/0    R+   11:33   0:00 ps au
```

To view process information, use the **ps** command. The default is to show only processes in the current shell. Add the **a** option to view all processes with a terminal. To view the user associated with a process, include the **u** option